

Shell Scripting

Antonio Pecchia

antonio.pecchia@unisannio.it

Dipartimento di Ingegneria
Laurea in Ingegneria Informatica
Sistemi Operativi



UNIVERSITÀ DEGLI STUDI
DEL SANNIO Benevento

Shell scripting

- Fino ad ora: comandi eseguiti tramite `shell`, ed utilizzo di meccanismi come pipe e redirectione;
- Nella pratica, può servire eseguire **più comandi** (al verificarsi di certe **condizioni**) o **iterarli**;
- Molte `shell`, tra cui **bash**, supportano un **linguaggio di scripting** (variabili, strutture di controllo, sintassi, *etc.*).

***NOTA:** la presentazione copre un insieme di funzionalità *base* della shell `bash` e dello scripting.

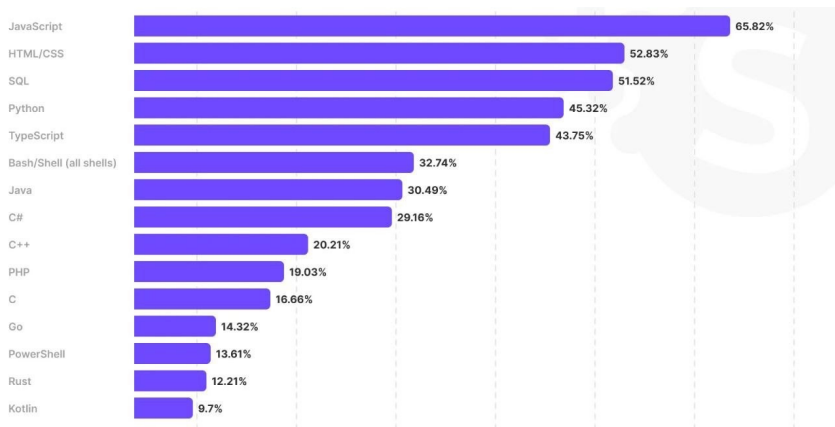
Shell scripting

- Linguaggio ottimizzato per eseguire task "*shell-related*":
 - creazione di pipeline di comandi;
 - cercare file/directory;
 - effettuare ricerche e manipolazioni testuali;
 - creare, cancellare, spostare directory e files;
 - gestire processi;
 - *automatizzare* task ripetitivi;
 - ...

tutto ciò che si può costruire sulla base dei tantissimi comandi e programmi di utilità!

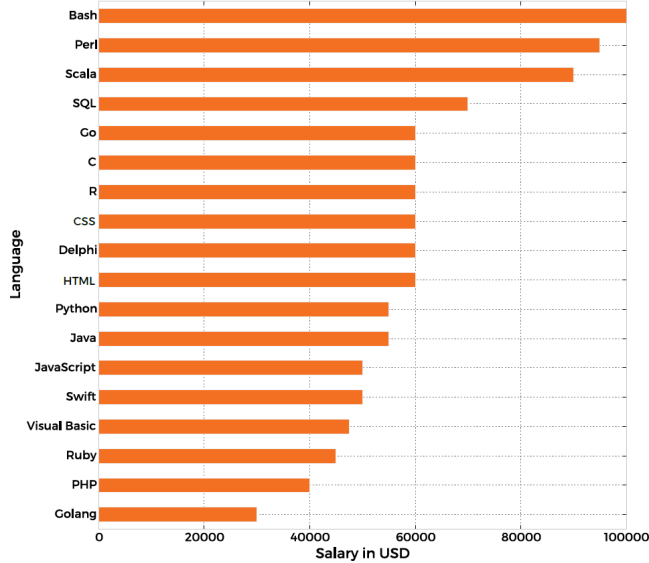


Exploring the Top Programming Languages and Frameworks Shaping 2024

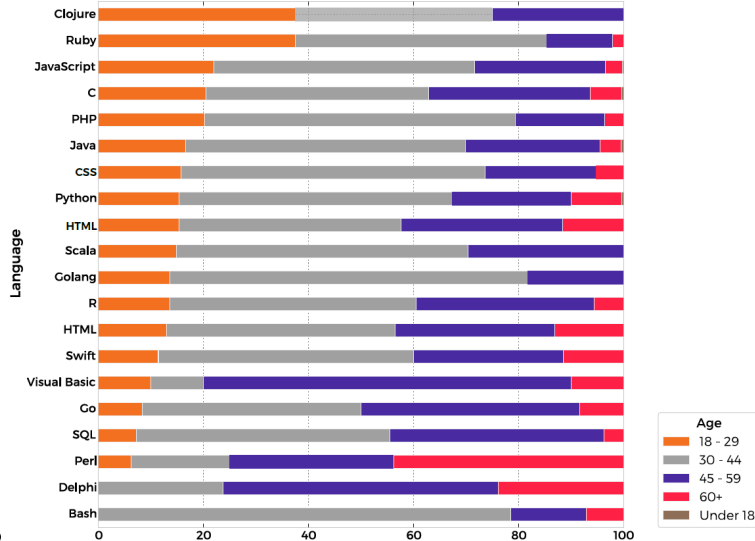


*Source: www.linkedin.com

Median stated salary by programming language



Programming languages age breakdown



*Source: adtmag.com

Linux Command Cheat Sheet

Basic commands	File management	File Utilities	Memory & Processes
 sudo [command] nohup [command] man [command] [command] & >> [fileA] > [fileA] echo -n xargs T>2& fg %N jobs ctrl-z	find ls -a -C -h rm -r -f locate -i cp -a -R -i du -s file -b -i mv -f -i grep, egrep, fgrep -i -v	tr -d uniq -c -u split -l wc -w head -n cut -s diff -q join -i more, less sort -n comm -3 cat -s tail -f	free -m killall sensors top kill -1 -9 service [start stop restart] ps aux dmesg -k
File permission	File compression	Scripting	Disk Utilities
chmod -c -R touch -a -t chown -c -R chgrp -c -R touch -a -t	tar xvfz gzip, gunzip, xcat uueencode, uuedecode zip, unzip -v rpm bzip2, bunzip2 rar	awk, gawk tsh -- .. python bash ksh php csh, tcsh perl source (file)	df -h, -l mkfs -t -V resize2fs fsck -A -N pvcreate mount -a -t fdisk -l lvcreate umount -f -v
Network	File Editor	Misc Commands	
netstat -r -v telnet tcpdump ssh -i ping -c	ex vi nano view emacs sublime sed pico	pwd -P bc expr cal export date -d \$(variable)	

source: Docsity

Compiled by A. Khoo

Alcuni comandi di base* (lezioni precedenti)

- **pwd** // print current directory;
- **ls** // list directory content;
- **cd** // change directory;
- **mkdir** // make directory;
- **rm** // remove files or directories (**rm -r**);
- **mv** // move - rename files;
- **cp** // copy files or directories;
- **ps aux** // snapshot of current processes.

* Per questi comandi e gli altri comandi in seguito, si ricorda l'utilizzo dei flag **-h** oppure **--help**, o anche del comando **man**.

Ricerca di file e directory: `find`

- La ricerca di file e directory è un task molto comune. Tutti i sistemi UNIX-like forniscono il comando `find`:
 - ricerca ricorsiva di file che soddisfano un criterio.

```
# Find all directories named src in .  
find . -name src -type d
```

```
# Find all txt files in the resources directory  
find resources/ -name "*.txt" -type f
```

NOTA: si noti l'utilizzo di `*` in `"*.txt"`, che indica un numero arbitrario di caratteri.

Ricerca di file e directory: `find`

- Oltre a ricercare file e directory, `find` può essere usato anche per **eseguire comandi** sui file trovati.

```
# Delete all files with .tmp extension  
find . -name '*.tmp' -exec rm {} \;
```

Visualizzazione righe (per es., di file)

□ Alcuni comandi: `cat`, `head`, e `tail`:

- `cat` // concatena file e mostra sullo standard output;
- `head` // mostra la prima parte di un file;
- `tail` // mostra la prima finale di un file.

```
$ cat test.txt
Linux
Bash
Bash
File
Bash
Bash
Test
Book
Book
Hello
```

```
$ head -n 3 test.txt
Linux
Bash
Bash
```

```
$ tail -n 4 test.txt
Test
Book
Book
Hello
```

Conteggio e ordinamento

□ Alcuni comandi: `wc`, `uniq`, e `sort`:

- `wc` // print newline, word, and byte counts;
- `uniq` // report or omit repeated lines;
- `sort` // sort lines of text files.

```
# Conta le line di words.txt
$ wc -l words.txt
30 words.txt
```

```
# Conta gli elementi (non nascosti) sotto resources
$ ls resources| wc -l
4
```

Conteggio e ordinamento

- Attenzione al funzionamento di **uniq** (input *non* ordinato):

\$ cat test.txt	\$ uniq test.txt	\$ uniq -c test.txt
Linux	Linux	1 Linux
Bash	Bash	2 Bash
Bash	File	1 File
File	Bash	2 Bash
Bash	Test	1 Test
Bash	Book	2 Book
Test	Hello	1 Hello
Book		
Book		
Hello		

NOTA: **uniq -c** produce anche un conteggio

Conteggio e ordinamento

- Soluzione: **sort** e **uniq**:

\$ cat test.txt	\$ uniq test.txt	\$ sort test.txt	\$ sort test.txt uniq
Linux	Linux	Bash	Bash
Bash	Bash	Bash	Book
Bash	File	Bash	File
File	Bash	Bash	Hello
Bash	Test	Book	Linux
Bash	Book	Book	Test
Test	Hello	File	
Book		Hello	
Book		Linux	
Hello		Test	

NOTA: **sort -n** ordina numericamente (anziché in maniera lessicografica); **sort -r** ordina in maniera inversa; **sort -nr**, numericamente ed in ordine decrescente.

Data handling

- ❑ **grep** stampa le righe che soddisfano (o non) un pattern:

\$ cat test.txt	\$ grep "Bash" test.txt	\$ grep -v "Bash" test.txt
Linux	Bash	Linux
Bash	Bash	File
Bash	Bash	Test
File	Bash	Book
Bash		Book
Bash		Hello
Test		
Book		
Book		
Hello		

NOTA: **grep** supporta le **espressioni regolari**, e la specifica di pattern complessi.

Data handling con awk

- ❑ **awk** è un *linguaggio* orientato alla manipolazione dati di tipo testuale (da file o standard input).
- ❑ Ha **molte** funzionalità: per semplicità, negli esercizi mostrati **awk** verrà usato "semplicemente" per isolare l' *n-esimo* token dalle righe di input.

Data handling con awk

- **Esempio 1:** dato l'input, mostra i campi indicati da `print`.

```
$ ps aux | head -n 4
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.0  0.1 169784 13116 ?        Ss   nov15   10:39 /sbin/init splash
root         2  0.0  0.0      0      0 ?        S    nov15    0:09 [kthreadd]
root         3  0.0  0.0      0      0 ?        I<   nov15    0:00 [rcu_gp]
```



```
$ ps aux | head -n 4 | awk '{print $5, $2}'
VSZ PID
169784 1
0 2
0 3
```

Data handling con awk

- **Esempio 2:** dato l'input, mostra i campi indicati da `print` se un certo campo verifica una condizione:
 - nell'esempio si mostrano USER, VSZ e RSS (campi \$1, \$5 e \$6 di `ps aux`) del solo processo con PID 154348 (in `ps aux` i PID sono in colonna \$2).

```
$ ps aux | awk '($2==154348) {print $1, $5, $6}'
studente 19532 5260
```

NOTA: se il confronto è con una stringa, essa va inclusa in `"..."` ; i confronti possono essere molto più complessi (per es., più campi e clausole AND–OR).

Esercizio

- Data in input la directory **resources**, si produca uno script **bash** che cerchi tutti i file **.txt** in essa presenti, e che salvi tali file in una directory specificata dall'utente.

Una possibile soluzione

```
#!/bin/bash

# utilizzo: ./backup.sh testdir

# $1 -> identifica il primo argomento, assegnato
# alla variabile BACKUPDIR
BACKUPDIR=$1

# rimuove la directory specificata nel caso questa esista
rm -rf $BACKUPDIR

# crea la directory specificata
mkdir $BACKUPDIR

echo "Inizio backup in $BACKUPDIR ..."

# cerca i file con estensione .txt e copia
# i file nella directory specificata

find resources/ -name "*.txt" -type f -exec cp {} $BACKUPDIR \;

# conta i file copiati e salva il conteggio nella variabile count

count=$(ls $BACKUPDIR | wc -l)

echo "Fine backup di $count files."
```

```
$ ./backup.sh mydir
Inizio backup in mydir ...
Fine backup di 8 files.
```

Spiegazione righe iniziali

- ❑ **shebang** (ossia i caratteri **#!**, collocati all'inizio di uno script), indica al sistema quale interprete utilizzare:
 - qualsiasi altra riga che inizi con **#** è **un commento**.
- ❑ È possibile passare argomenti ad uno script:

\$1, \$2, ... \$10, \$11 : are the values of the first, second
etc arguments

\$0 : The name of the script

Assegnare il risultato di un comando

- ❑ Un pattern tipico, è quello di assegnare il risultato di un comando (o di una pipe di comandi) ad una variabile:
- ❑ **\$(CMD)** : esegue il comando **CMD** e lo sostituisce con il proprio output. **Due alternative**:

```
var=$(command-name-here)
var=`command-name-here`
```

- ❑ Esempio:

```
$ wc -l linux.log
2000 linux.log
$ FILENAME=$(wc -l linux.log | awk '{print $2}')
$ echo $FILENAME
linux.log
```

Sidenote: quote, double quote, back quote

- **single quote** ('): inserire caratteri tra single quote, ne preserva il valore letterale;
- **double quote** ("): inserire caratteri tra double quote, ne preserva il valore letterale, **fatta eccezione** per \$ ' \ !
- **back quote** (`): esegue il comando tra i back quote, come **\$()**.

```
studente@opsys:~$ declare -i val=10
studente@opsys:~$ echo "Valore di val = $val"
Valore di val = 10
studente@opsys:~$ echo 'Valore di val = $val'
Valore di val = $val
studente@opsys:~$ val=`ls -la | wc -l`
studente@opsys:~$ echo "Valore di val = $val"
Valore di val = 30
```

Esercizi

- Modificare il precedente script **backup.sh** affinché sia possibile passare come argomento di input *anche* la directory sorgente. Esempio di invocazione:
./backup.sh srcdir dstdir
- Produrre uno script che ordini i PID dei processi correnti per RSS decrescente, e mostri a video i **top 5 PID** (l'*RSS* è il *resident set size* –memoria occupata dal processo in RAM– ed è il 6° campo mostrato da **ps aux**);
- Produrre uno script che, data la directory **resources**, cerchi il file **.txt** con il massimo numero di righe e ne mostri a video il nome.

ESEMPIO CON IF, WHILE E CONFRONTI (MATERIALE EXTRA)

Approfondimento

- ❑ Soluzione (non ottimale) finalizzata a mostrare un esempio d'uso dei costrutti **if** e **while**, previsti dal linguaggio bash.

```
$ ./search_copy.sh resources mydir
Cerco file .txt in resources
Trovati 7 file; creo mydir...
File copiati
```

```
$ ls mydir/
A_test.txt  fileA.txt  fileB.txt  file.txt
myfile.txt  testB.txt  testC.txt
```

```
#!/bin/bash

SRCDIR=$1
DSTDIR=$2

declare -i count=0

echo "Cerco file .txt in $SRCDIR"

find $SRCDIR -name "*.txt" -type f > list.txt

count=$(wc -l list.txt | awk '{print $1}')

if [ $count -eq 0 ]
then
    echo "Nessun file trovato!"
else
    echo "Trovati $count file; creo $DSTDIR..."

    mkdir $DSTDIR

    cat list.txt | while read NAME
    do
        cp $NAME $DSTDIR/
    done

    echo "File copiati"
fi

rm list.txt
```

Approfondimento: test di espressioni

- Confronto di 2 numeri (nello script della slide precedente è utilizzato `-eq`, ossia `==`);
- Altre possibilità (se $n1$ o $n2$ non sono numeri, il test fallisce):

`n1 -eq n2` : tests if $n1 = n2$

`n1 -ne n2` : tests if $n1 \neq n2$

`n1 -lt n2` : tests if $n1 < n2$

`n1 -le n2` : tests if $n1 \leq n2$

`n1 -gt n2` : tests if $n1 > n2$

`n1 -ge n2` : tests if $n1 \geq n2$